

PBS Survey

Gil Henkin

31.10.2025

Abstract

Proposer-Builder Separation (PBS) has become a foundational mechanism in Ethereum’s post-Merge architecture, aiming to mitigate Maximal Extractable Value (MEV) centralization and democratize block production. This paper surveys the evolution of PBS from its off-chain implementation through *MEV-Boost* to the emerging in-protocol design known as *Enshrined PBS* (ePBS). We provide an overview of the motivation behind PBS, its operational flow, and the associated trust assumptions between proposers, builders, and relays. The survey highlights key challenges such as censorship resistance, relay centralization, and proposer-builder collusion, then examines how recent research and proposals—including EIP-7732—seek to enshrine PBS within Ethereum’s consensus. Finally, we discuss the broader implications of these designs for decentralization, validator incentives, and protocol sustainability, identifying open questions and future research directions in enshrined block markets.

1 Introduction

In proof-of-stake Ethereum, validators (proposers) receive on-chain rewards for proposing blocks, but can earn significantly more by exploiting Maximal Extractable Value (MEV) from transaction ordering. To distribute MEV opportunities, Ethereum adopted Proposer-Builder Separation (PBS) via the MEV-Boost relay system. In MEV-Boost, validators outsource block construction to off-chain *builders* through relays who assemble high-value blocks, submit a sealed bid, and then deliver the full block after the proposer commits. This separation flattens the playing field among validators: proposers no longer need to run sophisticated builder algorithms, and instead accept the highest bid without seeing the transactions. However, the current PBS ecosystem introduces new trust and centralization issues.

Today, a small number of relays deliver the majority of blocks, creating a central point of failure. Builders must trust relays not to censor or steal blocks, and proposers must trust that builders will honor bids after the block is committed. Several recent works and proposals (e.g. EIP-7732) aim to enshrine PBS in-protocol.

2 Background and Related Work

This section presents the PBS solutions that have been proposed so far.

2.1 MEV and MEV-Boost

Maximal Extractable Value (MEV) refers to the extra gains block proposers or extract by reordering or including special transactions beyond base fees. In Proof-of-Stake (PoS) Ethereum, validators seek MEV by constructing blocks containing arbitrage, liquidation, and sandwich transactions. To level the field, PBS was introduced: flashbots' MEV-Boost allows validators to outsource block building to specialized builders via relays. Validators register interested relays and pass along builder proposals. This opens competition: over 90% of Ethereum validators now use MEV-Boost to bid out blockspace.

2.2 Proposer-Builder Separation (PBS)

PBS separates the roles of *building* a block (assembling transactions for maximal revenue) from *proposing* it to the consensus layer. Intuitively, block-building benefits from private information and computation. Under PBS, builders produce candidate blocks and offer to pay the proposer for their block's inclusion. The proposer accepts the highest bid among builders, then signs its header and publishes it. Conceptually, PBS aims to delegate MEV advantages to the *builder* side, so validators (proposers) simply ensure validity without needing the same MEV strategies. In the context of two entities - builder and proposer there are a few attacks that each of the party can leverage to exploit in the simple naive communication between the two parties.

2.3 Types of Different Attacks

In this subsection we list the problem of the naive setup - a system that is comprised of a proposer and a builder only (this is not a PBS design but rather the "naive" approach).

When one is observing PBS (as it is presented and working today) from the prescriptive of Ethereum's philosophy and decentralization mind set, it is an inherently wrong approach, thus it is interesting to observe the problem in the naive setup so that we can address the root problems, instead of patching the patch. Below we list the attacks:

Running Away with Fees

Proposer can run away with fee rewards as soon the block is known to him - by gossiping the block and collecting all the reward to himself without sharing with other participants.

Block Augmentation

Proposer can augment and modify the block to his own advantage by re-ordering the transactions or adding/removing transaction according to his own benefit to increase his own personal MEV (and not builders).

Block Delay

Block builder can claim that he has the block with the largest MEV and delay block transmission at once the proposer selects the block exhausting proposer slot time and straving the proposer such that he will not get nor block fees neither MEV.

Lying

There are two main challenges when it comes to fraud detection in this context. The first is to detect whether somebody is lying in this chain of communication, starting from the builder all the way to proposer, and the second problem is to detect who is lying and slash him accordingly.

The most simplest solution would involve economic incentives where the proposer and the builder would stake in order to participate in the protocol.

Stealing Blocks

Relays can steal builders block since they are fully exposed to block's content.

2.4 Naive Approach

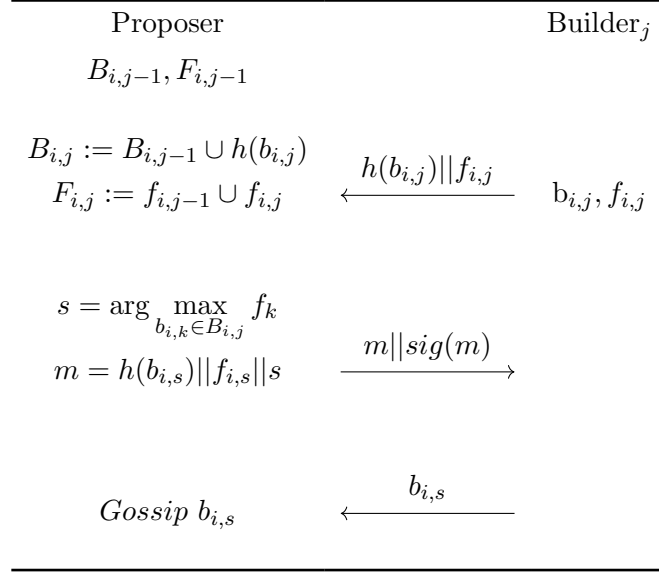
1 presents a naive variant of a PBS protocol, it does not try to prevent any attacks listed in 2.3, but rather presents a baseline. This protocol assumes full honest participation of both parties.

The flow of the protocol is as such:

Step 0 The proposer aggregates proposed block candidates for slot i of all builders so far.

Step 1 Once builder j evaluates a candidate, he submits $h(b_{i,j})$ - hash of the block candidate, to the proposer and the promised fee $f_{i,j}$. The proposer of slot i aggregates the header.

Step 2 The proposer selects a block candidate by signing on the correspond-



Protocol 1: Naive Proposer-Builder Separation

ing block header and submits it as a request to the builder.

Step 3 The relay receives the proposer’s selected header and response with block’s payload $b_{i,j}$.

Step 4 The proposer receives the selected block payload $b_{i,j}$, validates it, and gossips the block. Fees are embedded as a transaction within the block, rewarding the proposer, and validator once the block is finalized. The builder co-gossips the same block (signed by the proposer) to prevent any modification.

If the proposer attempts to broadcast an altered payload $b'_{i,j}$, the builder can prove equivocation by publishing the originally signed $b_{i,j}$, since both share the proposer’s signature but differ in hash. This results in a fork at slot i and triggers proposer slashing for producing two conflicting blocks. Because the proposer signs only the hash $h(b_{i,j})$ before receiving the payload, the builder’s co-gossip guarantees immutability: any divergence between $b_{i,j}$ and $b'_{i,j}$ is cryptographically verifiable and punishable.

2.5 PBS

Despite its benefits, current PBS/MEV-Boost is heavily based on trust assumptions. Proposers commit to a block only knowing that the block is valid and the payment amount, not the inside transactions. This eliminates the need to trust builders about ordering, but it still assumes that relays and builders behave correctly. In practice, a handful of relays deliver $\approx 90\%$ of the blocks, introducing a centralization point. Moreover, a builder may behave maliciously: for example, a builder might bid to win a slot and then fail to deliver the block (preventing the proposer to provide a valid block, and thus prevent the proposer to receive reward and potential MEV). In contrast, a relay might attempt to steal MEV by revealing or reassembling the block out of band. As illustrated in Figure 2.5, MEV-Boost orderflow connects searchers, builders, relays, and proposers through an external auction mechanism.

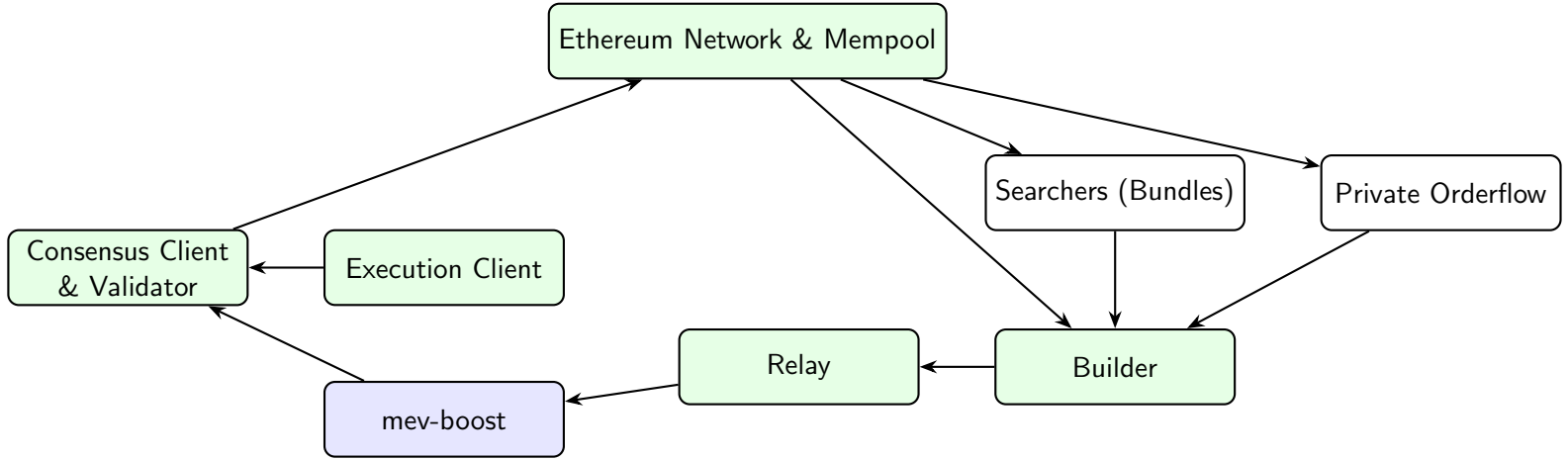


Figure 1: MEV-Boost Orderflow.

Several proposals have been made to address these issues. One avenue is encryption and fraud proofs. For instance, builders could encrypt blocks to the proposer's key so the relay cannot inspect them; fraud proofs can enable punishment if the builder withholds the block. Trusted Execution Environments (TEEs) have also been proposed (so-called TEE-Boost) to prove block validity to the proposer without revealing content. More recently, threshold encryption (encrypting to an attestation committee) has been shown to allow instant private block delivery once enough attestations

accumulate.

2.5.1 PBS Protocol

Protocol 2 presents today's version of PBS after the introduction of MEV-boost. In the protocol diagram, we combined the relay and the mev-boost entities into a single entity (for simplicity), a more detailed version of the diagram can be seen [1]. We focus on the high level design of block selection and assume that each block proposal happens after the registration phase.

In the protocol is comprised of 3 entities:

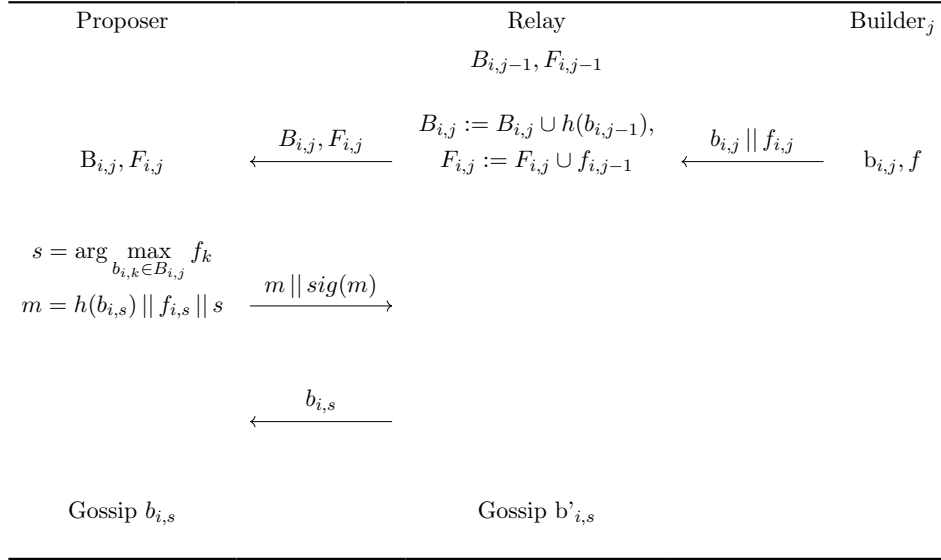
Proposer is a block proposer (and a staked validator) at i .

Relay Is a staked validator.

Builder Is selected by the proposer according to the reward fee that he guarantees to the proposer, the builder is not staked in PBS protocol.

The flow of the protocol is as such:

Step 0 The relay aggregates proposed block candidates for slot i of all



Protocol 2: PBS — Relay-Assisted Block Submission

builders so far.

Step 1 Once builder j evaluates a candidate, he submits to the relay, the candidate block $b_{i,j}$ and the promised fee $f_{i,j}$. The relay adds the new candidate to the collection and submits block header ($h(b_{i,j})$ - hash of the block candidate) and the promised fee $f_{i,j}$ to the designated block proposer of slot i .

Step 2 The proposer selects a block $b_{i,s}$ candidate by signing on the corresponding block header $sig(h(b_{i,s}))$ and submits it as a request to the relay.

Step 3 The relay receives the proposer's selected header and responds with block's payload $b_{i,j}$.

Step 4 The proposer receives the selected block payload $b_{i,j}$, and validates it.

Step 5 Proposer, gossips the block. Fees are embedded as a transaction within the block, rewarding the proposer, relay, and validator once the block is finalized. The relay co-gossips the same block (signed by the proposer) to prevent any modification.

2.5.2 What exploits does PBS solve?

Proposer-Builder Separation (PBS) is designed to mitigate several classes of manipulation and centralization risks that arise during block construction and propagation, such as those discussed in Section 2.3. Below we present the defense mechanisms that PBS introduce in order to deal with trust issue in the communication between proposers and builder communication:

If the proposer attempts to gossip an altered payload $b'_{i,j} \neq b_{i,j}$, the relay can prove equivocation by publishing the originally signed $b_{i,j}$, since both share the proposer's signature but differ in hash. This results in a fork at slot i and triggers proposer slashing for producing two conflicting blocks. Because the proposer signs only the hash $h(b_{i,j})$ before receiving the payload, the relay's co-gossip guarantees immutability: any divergence between $b_{i,j}$ and $b'_{i,j}$ is cryptographically verifiable and punishable. The co-gossip protection mechanism is the baseline defense for ??? most/all attacks that the naive protocol was vulnerable to. The PBS provides protection between the relay and the builder.

Running Away with Fees

The fees are included as a transaction in the block. The block can not be changed because the relay co-gossip the header signed by the proposer.

Block Augmentation

Block can not be changed because the relay co-gossip the header signed by the proposer.

Block Delay

The relay can delay block payload (step 4) in the protocol, and prevent the proposer from receiving mev reward as well as block inclusion reward. So this is still a valid attack that has not been addressed.

Lying

In the proposed protocol, the proposer can not lie, while the relay can change and augment blocks without being spotted. But he can not co-gossip wrong blocks since he receives only a single signature for a single block header.

Stealing Blocks

The relay can steal and reassemble the block he receives from the builders.

2.6 ePBS

Ethereum’s enshrined proposer-builder separation (ePBS) represents the ongoing effort to integrate the previously off-chain builder–proposer market directly into the consensus layer. The mechanism aims to preserve the decentralization and censorship-resistance of block production while formalizing the separation of responsibilities between builders, who construct optimized blocks, and proposers, who select and include them on-chain. ePBS introduces protocol-native commitments, bids, and cryptographic attestations to replace the current trust assumptions in MEV-Boost relays with verifiable guarantees. This shift is central to Ethereum’s roadmap for reducing centralization pressures, improving transparency in MEV extraction, and establishing a secure, in-protocol marketplace for block construction. As illustrated in 2.6, the ePBS orderflow connects searchers, builders and proposers through without the need for an external auction mechanism; eliminating completely the relay and mev-boost that PBS depended on, see 2.5.

Roles in the ePBS protocol:

Builders: these are validators that submit bids to proposers to commit them to a specific execution payload. Additionally, just like proposers, builders have to stake.

PTC: This is a new committee that attest for the timely presence (and validity) of the builder’s payload. The committee consists of 512 validators. The PTC vote for the timelines and validity of builder payload,

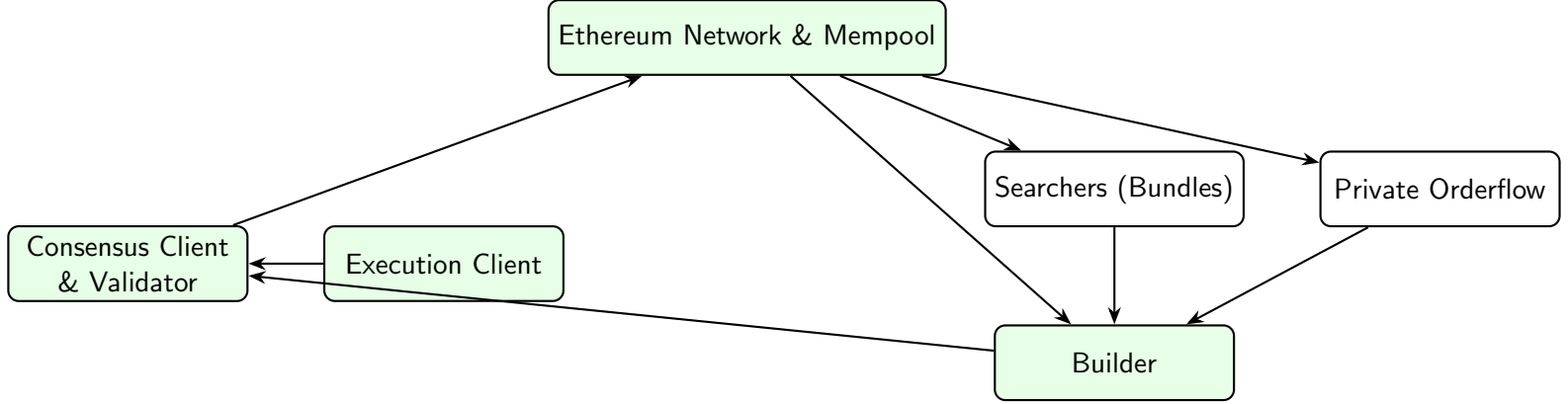


Figure 2: ePBS Orderflow.

where more than $2/3$ committee members have to vote for the block, thus a player that controls more than 35% of the stake.

then $1/3$ of the committee members can

The expected time for a malicious attacker, controlling 35% of the total stake, to have a majority control on the PTC is 205 000 years.

Proposer is guaranteed to be paid unconditionally.

The Payload Timeliness Committee members do not need to validate the execution payload before attesting to it. They perform basic checks such as verifying the builder's signature, and the correct blockhash is included. This takes away the full execution payload validation from the hot path of validation of an Ethereum block, giving the next proposer 6 seconds ($SECONDS_PER_SLOT * 2 // INTERVALS_PER_SLOT$) to validate the payload and every other validator 9 seconds ($SECONDS_PER_SLOT * 3 // INTERVALS_PER_SLOT$). From a user UX perspective, a transaction included in slot N by the builder is not widely validated until the proposer of slot $N + 1$ releases their beacon block on top of block N first and the attestors of slot $N + 1$ vote on this beacon block as the head of the chain, according to EIP-7732 specification [2].

The ePBS removes the relay and the mev-boost, instead it introduces Payload Timeliness Committee (PTC). In ePBS BeaconBlock and block's Payload are separated.

2.6.1 ePBS Protocol

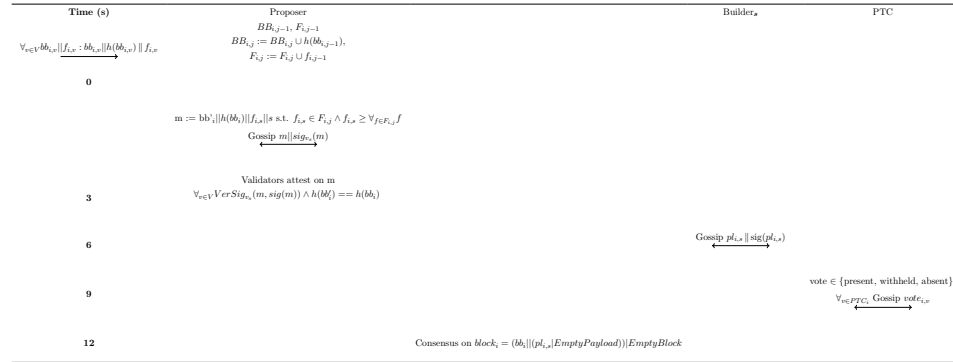
We present the ePBS protocol flow as explained in [3].

There are three entities in the ePBS protocol:

Roles.

- **Builder** — a (staked) validator under ePBS that signs a *SignedExecutionPayloadHeader* committing to the payload and later reveals the full payload; payment to the proposer is enforced on header processing.
- **Proposer** — the beacon-block author for the slot; selects a builder's signed header to include and receives the committed payment.
- **PTC validator** — a member of the *Payload Timeliness Committee* who attests whether the builder revealed the committed payload on time (and blob data availability per view), influencing fork choice.

the slot structure in Ethereum involves of ?? entities and evaluates as following:



Protocol 3: ePBS

Step 0, Second 0 At the beginning of the slot, the proposer assigned for that slot prepares and broadcasts a SignedBeaconBlock $BB_{i,s}$. In order to do so they first need to select a builder's bid $f_{i,s}$ and include the corresponding SignedExecutionPayloadHeader $h(BB_{i,s})$ in their beacon block.

Step 1, Between 0 and 3s During this time, the network has already gossiped the SignedBeaconBlock $m || \text{sig}_{v_s}(m)$ and the InclusionList (see 2.6.2). Validators will verify the proposer's signature of the SignedInclusionList-Summary and forward the full inclusion list for the EL to validate. Valida-

tors independently run the state transition function on the consensus block to validate it.

Step 2, Between 3 and 6 Validators attest for the presence of the corresponding beacon block and inclusion list. Validation of this block is fast at pre-merge times given that there is no execution to validate at this time of the slot $\forall_{v \in V} VerSig_{v_s}(m, sig(m)) \wedge h(bb'_{i,s}) == h(bb_{i,s})$. Propagation is also faster given that there are no blobs to be required at this stage and the block itself is lighter without the execution payload.

There is no new requirement for validators in what attestations are concerned.

Step 3, Between 6s and 9s Two things happen at the same time at half the slot time.

- Aggregators listening to their subnets aggregate and submit attestation aggregates.
- Builders broadcast their execution payload.

Nothing changes for aggregators except the tighter deadline from 8 seconds to 6 seconds.

Builders on the other hand have been watching all subnets and by 6 seconds should have a clear idea if the consensus layer block has been seen by a large portion of the network. Builders send their execution payload as an `SignedExecutionPayloadEnvelope` $pl_{i,s} \parallel sig(pl_{i,s})$ object.

Step 4, Between 9s and 12s Every slot, 512 validators are chosen to be part of a payload timeliness committee (PTC). Those validators attest, at 9 seconds into the slot if they have seen the expected execution payload timely. This committee can vote in three different ways

- If they have seen a consensus block for the current slot and the corresponding payload was seen timely with `payload_withheld = False` in the envelope, they vote payload present.
- If they have seen a consensus block for the current slot and the corresponding payload was seen timely with `payload_withheld = True` in the envelope, they vote payload withheld.
- If they have not seen any consensus block in the current slot, or if they have seen one but the corresponding payload was not seen they vote payload absent.

These attestations are gossiped on a global topic.

Step 5, At 12s By the end of the slot, validators have thus imported and validated an inclusion list, a consensus block, single bit attestations and aggregated attestations, payload attestations and a full execution payload. They can evaluate the new head of the blockchain. This can now be either of the three options:

- A full block, that is both the consensus block and the corresponding execution payload have been imported.
- An empty block, that is the consensus block was imported, but the execution payload was not revealed on time.
- A skipped slot, that is when the consensus block was imported.

2.6.2 Fork-choice enforced Inclusion Lists

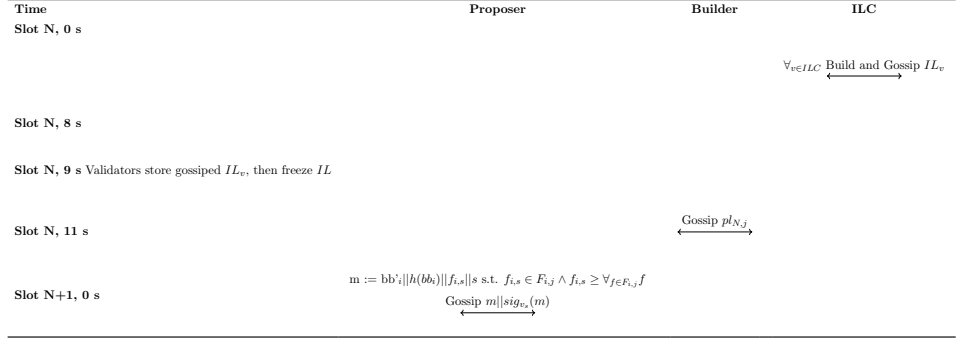
As discussed in [4], FOCIL (Fork-choice enforced Inclusion Lists) implements a robust mechanism to preserve Ethereum’s censorship resistance properties by guaranteeing timely transaction inclusion.

When validators receive inclusion lists (ILs) from the P2P network, they perform a series of validation checks before forwarding or caching them. These rules protect against Denial-of-Service attacks by (1) limiting ILs’ byte size and (2) restricting IL proposals to a small committee of IL committee members, thereby tightly bounding bandwidth, the main resource consumed by the propagation of ILs. Consumption of other relevant resources, such as verification time, is minimal because the only nontrivial check performed on IL propagation is signature verification. At this stage, there is no EL verification of the transactions within ILs. This means that ILs are allowed to contain any transactions—valid or invalid—since validators do not perform EL-side validity checks. This design choice is intended to avoid additional computational overhead.

Between Second 0-8 ILC (Inclusion List Committee) validator, construct their ILs by including transactions pending in the public mempool, up to 8Kib, where each validator can send two or less IL. Each ILC is of size 16 members.

Between Second 0-9 All the validators save gossiped IL, and freeze the IL afterwards.

Between Second 11 Payload $\forall_{j \in Builders} pl_{N,j}$ is gossiped by the block builders.



Protocol 4: FOCIL (EIP-7805) - Protocol Diagram. ePBS with Focil

Between Second 12 The proposer selects the block header with the highest fee and gossips the selected header, the rest of the protocol is follows the same order as the protocol presented in 3.

3 Discussion

End game? Or End of it? Will MEV kill Ethereum - or will Ethereum overcome its centralization bottlenecks and continue to push back against builders seeking to capitalize on its infrastructure and users?

This work aims to foster a broader and more competitive market of block builders, driving block-building margins down so that Ethereum’s users can benefit from a fairer, more transparent, and decentralized infrastructure.

Currently, a few builders dominate the block-building market [5], with Titan producing approximately 50% of all blocks. The slashing incentives must be carefully designed so that the optimistic mechanism enforces effective punishment in all cases - including scenarios where a single builder controls the majority of block production — while avoiding excessively high staking requirements that could lead to centralization.

References

- [1] “Flashbots mev-boost: Block proposal architecture overview.” <https://docs.flashbots.net/flashbots-mev-boost/architecture-overview/block-proposal>. Accessed: November 2025.
- [2] F. D’Amato, B. Monnot, M. Neuder, Potuz, and T. Tsao, “Eip-7732: Enshrined proposer-builder separation [draft].” <https://eips.ethereum.org/EIPS/eip-7732>, 2024. Ethereum Improvement Proposals, no. 7732, June 2024. [Online serial]. Accessed: November 2025.
- [3] Potuz, “Anatomy of a slot.” <https://hackmd.io/@potuz/rJ9GCnT1C#Anatomy-of-a-slot>. Accessed: November 2025.
- [4] T. Thiery, F. D’Amato, J. Ma, B. Monnot, T. Tsao, J. Kaufmann, and J. Song, “Eip-7805: Fork-choice enforced inclusion lists (focil) [draft].” <https://eips.ethereum.org/EIPS/eip-7805>, 2024. Ethereum Improvement Proposals, no. 7805, November 2024. [Online serial]. Accessed: November 2025.
- [5] “Relayscan: Real-time dashboard for ethereum mev-boost relays.” <https://www.relayscan.io/>. Accessed: November 2025.